

Анализ архитектурных ограничений GMock в контексте runtime expectations

Проблемы, демонстрация и предложение расширения

Sergey Dunaev

2026

Аннотация

Google Mock (GMock) - это широко используемый фреймворк для создания mock-объектов в C++. Одним из фундаментальных ограничений его архитектуры является невозможность изменения или условного применения **expectations** во время *runtime* теста. В этой работе будет проводится анализ внутренней архитектуры фреймворка *GMock* с точки зрения работы с expectations, продемонстрируются конкретные сценарии, в которых эти ограничения приводят к невалидным тестам или же сомнительным решениям-заглушка. Также, будет рассмотрено расширение его API, при помощи мною реализованного метода `When()`, который позволяет добавить *runtime-предикат* в цепочку проверки **expectations** с сохранением *полной* обратной совместимости со всеми методами этого тестового фреймворка.

Содержание

1	Мотивация	4
2	Введение	5
3	Архитектура GMock: хранение и механизм матчинга expectations	7
3.1	Макрос EXPECT_CALL()	7
3.2	Хранилище expectations	8
3.3	Потокобезопасность: для чего нужен g_gmock_mutex	8
3.4	Класс ExpectationBase: базовая структура любого expectation	9
3.5	Почему expectations иммутабельны	10
3.6	Алгоритм матчинга: метод FindMatchingExpectationLocked()	11

4	Проблемы runtime expectations: демонстрация	13
4.1	Проблема 1: матчеры фиксируются по значению	14
4.2	Проблема 2: expectation после фактического вызова	16
4.3	Проблема 3: условное ветвление в runtime	18
4.4	Итоги: ограничения GMock при тестировании runtime-логики	19
5	Поиски решения	20
5.1	Мотивация	20
5.2	Предлагаемое решение 1: метод .When()	21
5.2.1	Требования	21
5.2.2	Идея	21
5.2.3	Модификация исходного кода	21
5.2.4	Проблема верификации и её решение	22
5.2.5	Решение проблемы 3 с помощью .When()	23
5.2.6	Совместимость с существующим API	25
5.2.7	Плюсы такого подхода	26
5.3	Предлагаемое решение 2: State Machine	26
5.3.1	Мотивация	26
5.3.2	Модель	27
5.3.3	API-скетч	28
5.3.4	Решение исходных проблем	29
5.3.5	Совместимость с GTest	30
5.3.6	Ограничения и направления дальнейшего исследования . . .	30
6	Сравнительный анализ альтернативных фреймворков	31
6.1	Критерии сравнения	31
6.2	Trompeloeil	31
6.2.1	Общие сведения	31
6.2.2	Архитектура	32
6.2.3	Ключевые отличия синтаксиса	32
6.2.4	Проверка проблемы П1: runtime-значения	34
6.2.5	Проверка проблемы П2: гранулярность сброса	36
6.2.6	Проверка проблемы П3: условное ветвление	37
6.2.7	Возможность реализации .When() в Trompeloeil	39
6.2.8	Оценка по критериям	39
6.3	FakeIt	40
6.3.1	Архитектура	40
6.3.2	Синтаксис: сравнение с GMock	41
6.3.3	Проверка проблемы П1: runtime-значения в expectations . . .	43
6.3.4	Проверка проблемы П2: гранулярность сброса	45

6.3.5	Проверка проблемы ПЗ: условное ветвление	46
6.3.6	Возможность реализации .When() в FakeIt	48
6.3.7	Ограничения FakeIt	48
6.3.8	Оценка по критериям	49
7	Заключение	50

1. Мотивация

Современные системы всё чаще строятся вокруг асинхронной модели: рабочие потоки, колбэки, очереди событий. Тестирование таких систем на уровне юнит-тестов предполагает использование мок-объектов для изоляции компонентов. Однако де-факто стандартный C++ фреймворк мокирования – GMock – проектировался для синхронных детерминированных сценариев и плохо приспособлен для описания поведения, зависящего от runtime-состояния.

Эта проблема была обнаружена на практике при тестировании NetworkMonitor - асинхронного сервиса в составе проекта компании. Сервис имеет основной рабочий поток и несколько вспомогательных; последовательность и количество колбэков определяются в *runtime* и не могут быть предсказаны на этапе написания теста. GMock требует полного описания expectations до запуска тестируемого кода, что создаёт неразрешимое противоречие: тест вынужден предсказывать то, что по природе своей непредсказуемо.

Столкнувшись с этим ограничением, команда была вынуждена прибегнуть к обходным решениям: принудительная синхронизация потоков, эмуляция состояний, навязывание детерминированного порядка вызовов. По сути - подмена реального поведения системы искусственным. Такие тесты проходят, но проверяют не то, что происходит в production. Как следствие, значительная часть сценариев осталась **без** покрытия, что привело к серии дефектов, обнаруженных уже в эксплуатации. Проблема воспроизводится и в других сервисах со схожей асинхронной архитектурой; на момент написания статьи она остаётся нерешённой.

Интеграционные тесты частично закрывают этот пробел, однако их написание трудоёмко, а принцип пирамиды тестирования предписывает, чтобы основной объём покрытия обеспечивался юнит-тестами - быстрыми, изолированными и дешёвыми в поддержке.

Данная работа ставит целью: (1) систематически выявить и описать архитектурные ограничения GMock, препятствующие тестированию асинхронного кода; (2) оценить, решают ли эти проблемы альтернативные C++ фреймворки мокирования; (3) предложить минимальное расширение, устраняющее обнаруженные ограничения.

2. Введение

GTest и GMock являются де-факто стандартными фреймворками для тестирования и mock-объектов в экосистеме C++. Важно отметить, что изначально GTest задумывался сугубо как фреймворк для модульного тестирования. Почему это важно? - Станет понятно чуть позже.

GMock позволяет создавать mock-объекты при помощи макроса `MOCK_METHOD` и декларативно задавать `expectations` через `EXPECT_CALL()`.

Обычный тест с использованием GMock выглядит следующим образом:

Listing 1: Базовый пример использования GMock

```
1 class MockTransport : public ITransport {
2 public:
3     MOCK_METHOD(bool, connect, (int session_id), (override)); // Замокировали метод
4 };
5
6 TEST(Example, BasicUsage) {
7     MockTransport transport;
8     EXPECT_CALL(transport, connect(42))
9         .Times(1)
10        .WillOnce(Return(true));
11    // Декларативно задали ожидание вида: если у класса transport будет вызван
    метод connect() с параметром 42, то один раз верни true
12
13    bool result = transport.connect(42);
14    EXPECT_TRUE(result);
15 }
```

Данный пример, хоть и упрощенно, но достаточно хорошо иллюстрирует основные преимущества этого тестового фреймворка.

А именно, простота для любого читателя(маломальски знакомого с английским) и в то же время очень обширный набор инструментов для разработчика. Понятно, что такая модель написания тестов отлично подходит для модульных тестов, а также для каких-то *синхронных* и *детерминированных* сценариев, где у теста есть чёткая последовательность действий и заранее известный ожидаемый результат.

Однако в реальном мире все далеко не так радужно. В подавляющем большинстве современных проектов есть множество мест, где активно используется асинхронный стиль написания кода, с колбеками и различными ветвлениями из условий на основе *runtime*-логики тестируемого модуля, компонента. Отсюда закономерным образом вытекает пробелема с тестированием подобных сценариев, которые зависят от недетерминированных условий *runtime*.

Иными словами, возникает *фундаментальная проблема* - по логике фреймворка, все

expectations должны быть полностью определены *до начала* выполнения тестируемого кода, это означает, что тест должен *предсказать все* runtime-решения заранее, что противоречит самой природе тестирования условной логики асинхронного кода.

Как следствие такой несовместимости стилей и необходимости хоть как-то тестировать подобного рода код, на свет рождаются решения сомнительного качества, среди которых я находил следующие:

1. **Детерминированные эмуляторы асинхронной среды**¹. Вокруг тестируемого модуля выстраивается синхронная обёртка, имитирующая асинхронное поведение в строго заданном порядке. Такие эмуляторы трудоёмки в написании и поддержке, а главное - они не тестируют реальное поведение системы: гонки, произвольный порядок колбэков и таймауты остаются за пределами проверки.
2. **Тесты с фиксированными задержками**. Между действием и проверкой вставляется пауза (`std::this_thread::sleep_for`), достаточная, чтобы асинхронная операция «успела» завершиться. Такие тесты крайне не стабильные: на слабой машине или под нагрузкой CI задержки оказываются недостаточно и тест падает, а увеличение паузы замедляет весь набор тестов.
3. **Отказ от использования моков**. Вместо изолированного тестирования модуля, пишется практически реальная инфраструктура для написания интеграционного теста. Такой подход безусловно решает проблему достоверности, но цена такого подхода: большое время разработки, долгая скорость исполнения и воспроизводимость.

Каждый из перечисленных подходов по-своему решает проблему, но ни один не устраняет её первопричину - статическую природу *expectations* в GMock. Вместо того чтобы обходить ограничение фреймворка, логичнее поставить вопрос так: «Можно ли расширить сам GMock так, чтобы *expectations* учитывали *runtime* состояния и при этом не нарушали совместимость с уже существующими тестами или же проще все бросить и написать все с нуля?»

В настоящей работе мы:

1. Подробно проанализируем внутреннюю архитектуру GMock: хранение *expectations*, алгоритм работы матчинга и модель подстаноки/верификации.

¹Наиболее известная промышленная реализация описана в: Zhou J. et al. *FoundationDB: A Distributed Key-Value Store* // Proc. ACM SIGMOD, 2021. <https://www.foundationdb.org/files/fdb-paper.pdf>. Аналогичный подход для Rust: <https://s2.dev/blog/dst>.

2. Продемонстрируем несколько конкретных сценариев, в которых из-за невозможности выставить *runtime-expectations* случается провал тестов или же потеря точности/валидности проверки.
3. Предложим минимальный совместимый *workaround* - метод `.When()`, и проанализируем его совместимость с существующей кодовой базой тестов.
4. Предложим более радикальную идею - написание альтернативного тестового фреймворка. Проанализируем и оценим потенциальные плюсы, минусы и риски.

3. Архитектура GMock: хранение и механизм матчинга expectations

Для понимания природы ограничений GMock необходимо рассмотреть, как он хранит и обрабатывает *expectations* на уровне исходного кода.

Все ссылки приводятся для версии *GoogleTest 1.15.0*.²

3.1. Макрос EXPECT_CALL()

Наверное самый фундаментальный макрос, `EXPECT_CALL()` определён в файле `gmock-spec-builders.h`³ и имеет следующий вид:

Listing 2: Определение `EXPECT_CALL()`

```
#define EXPECT_CALL(obj, call) \  
    GMOCK_ON_CALL_IMPL_(obj, InternalExpectedAt, call)
```

Дальше идет еще небольшой слой из более технического макроса `GMOCK_ON_CALL_IMPL_()`, который вызывает метод `gmock_##call` у `mock`-объекта. Он в свою очередь возвращает объект типа `MockSpec<F>` - основного строительного блока, вокруг которого и ведется основная часть работы этого фреймворка.

Затем вызывается метод `InternalExpectedAt()` у `MockSpec`, который создаёт новый `TypedExpectation<F>` и добавляет его в *хранилище*.

Важно отметить, что для аналогичного макроса `ON_CALL()` логика раскрытия макросов будет аналогична. Поэтому для краткости этот макрос мы рассматривать не будем.

²Исходный код: <https://github.com/google/googletest/tree/v1.15.0>. Основной файл, в котором сосредоточена логика expectations: `googletest/include/gmock/gmock-spec-builders.h`.

³Строка 353 в версии 1.15.0: <https://github.com/google/googletest/blob/v1.15.0/googletest/include/gmock/gmock-spec-builders.h#L353>.

3.2. Хранилище expectations

Каждый `FunctionMocker<F>` является *единственным* на каждый метод. Он содержит вектор⁴:

Listing 3: Вектор expectations в `UntypedFunctionMockerBase` (базовый класс `FunctionMocker`) (строка 205–226)

```
1 using UntypedExpectations = std::vector<std::shared_ptr<ExpectationBase>>;
2 ...
3 UntypedExpectations untyped_expectations_;
```

Сразу отметим, что это обычный `std::vector`, а соответственно *все* новые expectations добавляются через `push_back()`. Это происходит в методе `AddNewExpectation()`.

Listing 4: `AddNewExpectation()` класса `FunctionMocker`

```
1 TypedExpectation<F>& AddNewExpectation(const char* file, int line,
2                                     const std::string& source_text,
3                                     const ArgumentMatcherTuple& m)
4 {
5     Mock::RegisterUseByOnCallOrExpectCall(MockObject(), file, line);
6     TypedExpectation<F>* const expectation =
7         new TypedExpectation<F>(this, file, line, source_text, m);
8     const std::shared_ptr<ExpectationBase> untyped_expectation(expectation);
9     untyped_expectations_.push_back(untyped_expectation);
10    ...
11    return *expectation;
12 }
```

Принципиально важно отметить: доступ к вектору **не защищён мьютексом**. Добавление expectations из другого потока во время вызовов мока является UB и незапланированным сценарием использования.⁵

3.3. Потокобезопасность: для чего нужен `g_gmock_mutex`

Если вектор expectations не защищён мьютексом, возникает закономерный вопрос: а зачем тогда в коде GMock повсеместно встречается `g_gmock_mutex`⁶?

Ответ в том, что мьютекс защищает **не сам вектор expectations**, а **мутабельные поля** внутри каждого expectation, которые изменяются *во время* работы теста:

⁴Класс `UntypedFunctionMockerBase`, строки 205–226: <https://github.com/google/googletest/blob/v1.15.0/googlemock/include/gmock/gmock-spec-builders.h#L205>.

⁵Документация GMock явно предупреждает: «Adding a new expectation after a mock function has already been called [...] is **undefined behavior**», см. https://google.github.io/googletest/gmock_for_dummies.html#using-mocks-in-tests.

⁶Объявлен в `gmock-spec-builders.cc`, строка 54: <https://github.com/google/googletest/blob/v1.15.0/googlemock/src/gmock-spec-builders.cc#L54>.

- `call_count_` - счётчик фактических вызовов. Каждый раз, когда мок-метод вызывается из рабочего потока, GMock инкрементирует этот счётчик у найденного `expectation`.
- `retired_` - флаг деактивации. Устанавливается, когда `expectation` насыщен.
- `action_count_checked_` - флаг однократной проверки количества действий.

Если тестируемый код вызывает мок-метод одновременно из нескольких потоков (что часто бывает в асинхронных сценариях), то без мьютекса два потока могли бы одновременно читать и записывать `call_count_`, что привело бы к гонке данных.

Таким образом, модель потокобезопасности GMock выглядит следующим образом:

1. **Setup-фаза** (однопоточная): тест создаёт мок-объект, вызывает `EXPECT_CALL()`, заполняя вектор `untyped_expectations_`. Синхронизация не нужна - работает один поток.
2. **Фаза исполнения** (многопоточная): тестируемый код вызывает мок-методы из произвольных потоков. Вектор `expectations` только *читается* (обход в `FindMatchingExpectationLocked`), но мутабельные поля найденного `expectation` *модифицируются* - для этого и захватывается `g_gmock_mutex`.
3. **Фаза верификации** (однопоточная): при уничтожении мока GMock проходит по всем `expectations` и проверяет `call_count_` относительно `cardinality_`.

Именно это разделение на фазы объясняет, почему вектор не защищён мьютексом: авторы GMock *гарантируют* корректность только при условии, что все `EXPECT_CALL()` выполнены до начала вызовов мока. Нарушение этого контракта (например, добавление `expectation` из колбэка) является *undefined behavior*.

3.4. Класс `ExpectationBase`: базовая структура любого `expectation`

Класс `ExpectationBase`⁷ (`gmock-spec-builders.h`) содержит следующие ключевые поля:

- `cardinality_` - нужен, для работы макроса `.Times()`. Позволяет установить допустимый диапазон количества вызовов метода.
- `immediate_prerequisites_` - необходимы для работы макросов `InSequence` или `.After()`. Представляет из себя множество *expectations*, которые должны быть удовлетворены до вызова данного.

⁷Строки 670–870: <https://github.com/google/googletest/blob/v1.15.0/googlemock/include/gmock/gmock-spec-builders.h#L670>.

- **call_count_** - счётчик фактических вызовов метода, необходим для корректной проверки насыщенности всех ожиданий (того, что каждый установленный `EXPECT_CALL()` выполнится.)
- **retired_** - одноразовый флаг деактивации. Устанавливается, после того, как необходимое *expectations* отработало.
- **untyped_actions_** - вектор одноразовых действий. Нужен для работы макроса `WillOnce`
- **repeated_action_** - действие по умолчанию. Необходим для работы макроса `WillRepeatedly`.

Важно отметить, что все перечисленные поля, за исключением `call_count_` и `retired_`, являются **иммутабельными** после завершения `builder`-цепочки. Матчеры фиксируются *по значению* в момент вызова `EXPECT_CALL()`, а не по ссылке.

3.5. Почему *expectations* иммутабельны

Иммутабельность *expectations* после создания - это не произвольное техническое решение, а следствие модели верификации, лежащей в основе GMock.

GMock реализует паттерн «record-then-verify»⁸: сначала записываются *expectations*, так называемая *setup-фаза*, затем запускается тест, а затем GMock верифицирует результаты. Важно уточнить, что верификация происходит не только в деструкторе `mock`-объекта. GMock проверяет *expectations* в **двух** точках:

- **В момент вызова мока.** Если `mock`-метод вызван с аргументами, для которых не нашлось подходящего *expectation*, или если *expectation* уже насыщен (например, `Times(2)`, а вызов третий), GMock немедленно сигнализирует об ошибке. То есть *лишние* вызовы и вызовы с неожиданными параметрами обнаруживаются сразу, во время исполнения теста.
- **В деструкторе `mock`-объекта.** Здесь проверяется *обратная* ситуация: все ли *expectations* были удовлетворены. Если объявлено `Times(2)`, а метод был вызван только один раз, деструктор сообщит об ошибке «Expected: to be called twice. Actual: called once.»

Таким образом, *oversaturation* и вызовы с неожиданными аргументами ловятся мгновенно, а *undersaturation* - только при уничтожении мока.

⁸В документации GMock этот паттерн описан в разделе «Using Mocks in Tests»: https://google.github.io/googletest/gmock_for_dummies.html#using-mocks-in-tests.

Каждый *expectation* - это *константное утверждение*, сделанное в конкретный момент: «я ожидаю X вызовов с такими аргументами».

(На этой особенности важно заострить внимание, так как в силу нее дальнейшее решение проблемы будет иметь определенные, пусть и минимальные, но оговорки)

Если бы *expectations* были мутабельны, то сразу бы возник ряд вопросов связанных в первую очередь с верификацией исполнения моков.

В сухом остатке, такой подход в первую очередь гарантирует четкую структуру, гарантию верификации мок-объекта, а также **упрощает потокобезопасность**, так как если поля не меняются после *setup*-фазы, их можно читать во время работы теста **без** синхронизации.

В частности, если бы *expectations* допускали мутацию после *setup*-фазы, потокобезопасность резко усложнилась бы следующими вещами:

- **Рекурсивный лок.** Текущий `g_gmock_mutex` захватывается в `FindMatchingExpectationLocked` при каждом вызове мока. Если внутри действия (*action*) *expectation* разрешить модификацию другого *expectation*, потребуется рекурсивный мьютекс (`std::recursive_mutex`), что само по себе является признаком проблемного дизайна блокировок и потенциальной россыпью проблем, связанных с дедлоком.
- **Deadlock при нескольких объектах синхронизации.** В реальных сценариях мок-объект `Foo` может вызывать колбэк, модифицирующий *expectations* мок-объекта `Bar`, и наоборот. При наличии двух и более мьютексов возникает классическая проблема упорядочения блокировок и риск взаимной блокировки (*deadlock*).
- **Date-race между матчингом и мутацией.** Пока `FindMatchingExpectationLocked` обходит цепочку *expectations*, другой поток мог бы изменить матчер или кардинальность одного из них. Это требует либо копирования *expectations* перед обходом, либо удержания лока на всё время матчинга, что сериализует все вызовы моков и сводит на нет преимущества многопоточного тестирования.

Таким образом, иммутабельность *expectations* - это не только упрощение реализации, но и *сознательный* архитектурный выбор, позволяющий GMock оставаться пригодным для многопоточных тестов без сложных схем блокировок.

3.6. Алгоритм матчинга: метод `FindMatchingExpectationLocked()`

Разберем более детально краеугольный камень работы всего GMock, а именно алгоритм матчинга *expectation-ов*.

При вызове мокируемого метода фреймворк ищет подходящий *expectation* в методе `FindMatchingExpectationLocked()`⁹:

Listing 5: Алгоритм поиска matching expectation

```
1 TypedExpectation<F>* FindMatchingExpectationLocked(  
2     const ArgumentTuple& args) const {  
3     g_gmock_mutex.AssertHeld();  
4     for (typename UntypedExpectations::const_reverse_iterator it =  
5         untyped_expectations_.rbegin();  
6         it != untyped_expectations_.rend(); ++it) {  
7         TypedExpectation<F>* const exp =  
8             static_cast<TypedExpectation<F>*>(it->get());  
9         if (exp->ShouldHandleArguments(args)) {  
10            return exp;  
11        }  
12    }  
13    return nullptr;  
14 }
```

Как видно из листинга, код достаточно тривиален - мы попросту берем итератор на начало массива ожиданий и идем до ближайшего совпавшего с нашими аргументами(*args*). Важно отметить два аспекта:

1. **Обратный обход**: последний добавленный *expectation* проверяется первым. Это даёт семантику **LIFO**¹⁰, которая играет ключевую роль: она позволяет **уточнять** поведение мока без удаления ранее объявленных *expectations*. Общий *expectation* объявляется первым и служит поведением по умолчанию, а более специфичные *expectations* добавляются позже и перехватывают соответствующие вызовы, поскольку проверяются раньше. Например:

```
1 // широкий "expectation" - поведение по умолчанию  
2 EXPECT_CALL(mock, Send(_)).WillRepeatedly(Return(false));  
3 // узкий "expectation" - перехватывает конкретный аргумент  
4 EXPECT_CALL(mock, Send(42)).WillOnce(Return(true));
```

При вызове `Send(42)` сработает второй *expectation*; при вызове `Send(7)` – первый. Без LIFO-порядка пришлось бы явно исключать значение 42 из широкого *expectation*, что привело бы к громоздким и хрупким тестам.

⁹Строка 1684: <https://github.com/google/googletest/blob/v1.15.0/googlemock/include/gmock/gmock-spec-builders.h#L1684>.

¹⁰Документация GMock: «When there are multiple `EXPECT_CALL()`s [...] the newer takes precedence over the older» — https://google.github.io/googletest/gmock_for_dummies.html#MultiExpectations.

2. Проверка через `ShouldHandleArguments()`: для каждого *expectation* вычисляется конъюнкция условий. Далее рассмотрим этот метод подробнее.

Метод `ShouldHandleArguments()`¹¹ реализует следующую логику:

Listing 6: `ShouldHandleArguments` – оригинальная версия

```
1 bool ShouldHandleArguments(const ArgumentTuple& args) const {
2     g_gmock_mutex.AssertHeld();
3     CheckActionCountIfNotDone();
4     return !is_retired() // проверка того, что условие еще не было использовано
5           && AllPrerequisitesAreSatisfied() // специальный флаг для InSequence
6           && Matches(args); // метчатся ли аргументы
7 }
```

То есть:

- `is_retired()` - проверка флага деактивации;
- `AllPrerequisitesAreSatisfied()` - все *expectations* из последовательности пред-условий удовлетворены;
- `Matches(args)` - аргументы вызова соответствуют матчерам.

Отметим, что *ни одна из этих проверок* не зависит от внешнего состояния. То есть все проверки происходят сугубо изолированно, никак не опираясь на текущее состояние программы.

Уточнение: механизмы *InSequence* и *After*¹² формально проверяют очерёдность вызовов, то есть учитывают результат предыдущих операций – был ли предшествующий *expectation* уже удовлетворён. Однако эта проверка тоже носит **декларативный** характер: порядок фиксируется на этапе `setup` и не может быть изменён в `runtime`. *InSequence* лишь помечает *expectation* как `is_retired()` после насыщения, а `AllPrerequisitesAreSatisfied()` проверяет эти флаги. Никакой внешней логики вида «если операция X вернула ошибку – ожидать другой набор вызовов» эти механизмы выразить не позволяют.

Именно это делает **невозможным** условное применение *expectations* в зависимости от результатов предыдущих операций.

4. Проблемы `runtime expectations`: демонстрация

После небольшого погружения в логику работы механизма *expectation-ов*, мы наконец-то готовы погрузиться в конкретные проблемные сценарии, которые

¹¹Строка 1128: <https://github.com/google/googletest/blob/v1.15.0/googlemock/include/gmock/gmock-spec-builders.h#L1128>.

¹²Описание механизмов упорядочения: https://google.github.io/googletest/gmock_cook_book.html#OrderedCalls и https://google.github.io/googletest/gmock_cheat_sheet.html#AfterClause.

влечет за собой такая парадигма.

Для демонстрации будем использовать модель асинхронного сервиса с рабочим потоком, колбэками и условной логикой. Интерфейсы определены следующим образом:

Listing 7: Интерфейсы тестируемой системы

```
1 class IEventObserver {
2 public:
3     virtual ~IEventObserver() = default;
4     virtual void onConnected(int session_id) = 0;
5     virtual void onDataReceived(int session_id,
6                                 const std::string& data) = 0;
7     virtual void onDisconnected(int session_id, int reason_code) = 0;
8     virtual void onError(int session_id,
9                           const std::string& error) = 0;
10 };
11
12 class ITransport {
13 public:
14     virtual ~ITransport() = default;
15     virtual bool send(int session_id,
16                      const std::string& data) = 0;
17     virtual bool connect(int session_id) = 0;
18     virtual void disconnect(int session_id) = 0;
19 };
```

AsyncService содержит рабочий поток, обрабатывающий задачи из очереди. При вызове `postConnect(id)` сервис в рабочем потоке вызывает `transport.connect(id)` и, в зависимости от результата, уведомляет наблюдателя через `onConnected` или `onError`. На таком простом примере асинхронного сервиса мы и будем в дальнейшем разбирать все проблемы.

Все оригиналы кода из примеров можно найти в репозитории приложенном к этой статье.

4.1. Проблема 1: матчеры фиксируются по значению

Тестовый сценарий: сервис подключается, получает идентификатор сессии динамически, и дальнейшие *expectations* должны зависеть от этого идентификатора.¹³

¹³В листинге используется тип `testing::Expectation` - это type-erased handle GMock, предназначенный именно для передачи в `.After()`. См. https://google.github.io/googletest/gmock_cook_book.html#OrderedCalls.

Listing 8: Полная декларативная попытка с After

```

1  TEST(RuntimeExpectationsFailing,
2      DynamicId_ExpectationFrozenAtSetup_FAILS) {
3      MockObserver observer;
4      MockTransport transport;
5
6      int captured_id = -1;
7
8      // (1) Полный декларативный сценарий с упорядочением:
9      Expectation conn = EXPECT_CALL(observer, onConnected(_))
10         .WillOnce(SaveArg<0>(&captured_id));
11
12      EXPECT_CALL(transport, connect(_))
13         .WillOnce(Return(true));
14      EXPECT_CALL(transport, send(_, _))
15         .WillOnce(Return(true));
16
17      // (2) After(conn) гарантирует, что этот expectation
18      //      активируется только ПОСЛЕ onConnected.
19      //      Но captured_id == -1 в момент объявления!
20      //      GMock запоминает ЗНАЧЕНИЕ -1, а не ссылку.
21      EXPECT_CALL(observer, onDataReceived(captured_id, _))
22         .Times(1)
23         .After(conn);
24      //
25      //      == Eq(-1), хотя в runtime будет 42
26
27      AsyncService service(observer, transport);
28      service.start();
29      service.postConnect(42);
30      std::this_thread::sleep_for(
31          std::chrono::milliseconds(100));
32      service.postSend(42, "hello");
33      std::this_thread::sleep_for(
34          std::chrono::milliseconds(100));
35      service.stop();
36
37      EXPECT_EQ(captured_id, 42); // пройдёт, но тест
38                               // уже упал выше
39 }

```

Результат: тест падает с ошибкой <<Unexpected mock function call: onDataReceived(42,

"ack:hello")>>, несмотря на наличие `.After(conn)`. Механизм `After` корректно обеспечил порядок — `expectation` на `onDataReceived` стал активен только после срабатывания `onConnected`. Но матчер по-прежнему содержит замороженное значение `-1`, а фактический вызов приходит с `42`.

Причина: матчер `Eq(captured_id)` (подставляемый неявно) вычисляет и запоминает значение аргумента при создании, а не при каждой проверке. Это эквивалентно `Eq(-1)`.

Естественный вопрос: поможет ли `std::ref(captured_id)`? Нет. `EXPECT_CALL` раскрывается в шаблонный код, который конструирует матчер `Eq<int>`. Даже если передать `std::ref(captured_id)`, объект `std::reference_wrapper<int>` будет неявно преобразован в `int` при инстанцировании матчера, и значение всё равно скопируется. GMock-матчеры хранят **значение** типа `T`, а не ссылку на него.

Этот пример демонстрирует ключевое наблюдение: `After` и `InSequence` управляют *порядком активации* `expectations`, но не влияют на *содержимое* матчеров. Порядок и значения — два независимых аспекта, и декларативный подход GMock позволяет контролировать только первый.

В многопоточных тестах эта проблема усугубляется. Когда порядок и количество вызовов непредсказуемы (зависят от планировщика потоков, сетевых задержек, таймеров), декларативное описание последовательности через `InSequence`/`After` становится невозможным: нельзя заранее указать, что `onDataReceived` будет вызван ровно `N` раз после `onConnected`, если `N` определяется в `runtime`. В результате тесты вырождаются:

```
1 // Без After, без InSequence, без точного Times ---
2 // единственный способ не упасть на непредсказуемом
3 // количестве вызовов:
4 EXPECT_CALL(observer, onDataReceived(_, _))
5     .WillRepeatedly(Return());
```

Такой `expectation` по сути отключает проверку: он принимает любые аргументы, любое количество раз, без упорядочения. Тест компилируется и проходит, но перестаёт проверять что-либо содержательное. Это прямое следствие того, что GMock не предоставляет механизма для описания `expectations`, зависящих от *runtime-состояния*.

4.2. Проблема 2: `expectation` после фактического вызова

Сценарий: нам необходимо ставить *expectations* поэтапно: завершить одну фазу, затем на основании результатов объявить ожидания для следующей. Естественная идея — выставить `expectation` на `onDataReceived` прямо из колбэка `onConnected` через `Invoke`, когда `session_id` уже известен:

Listing 9: Попытка поставить expectation из Invoke

```

1  TEST(RuntimeExpectationsFailing,
2      PhaseExpectation_ViaInvoke_FAILS) {
3      MockObserver observer;
4      MockTransport transport;
5
6      EXPECT_CALL(transport, connect(_))
7          .WillOnce(Return(true));
8      EXPECT_CALL(transport, send(_, _))
9          .WillOnce(Return(true));
10
11     // Пытаемся выставить ожидание по "месту":
12     // внутри Invoke session_id уже известен.
13     EXPECT_CALL(observer, onConnected(_))
14         .WillOnce(Invoke([&](int session_id) {
15             // К этому моменту session_id == 42.
16             // Но EXPECT_CALL выполняется внутри
17             // g_gmock_mutex, который уже захвачен!
18             EXPECT_CALL(observer,
19                 onDataReceived(session_id, _))
20                 .Times(1);
21         }));
22
23     AsyncService service(observer, transport);
24     service.start();
25     service.postConnect(42);
26     std::this_thread::sleep_for(
27         std::chrono::milliseconds(100));
28     service.postSend(42, "hello");
29     std::this_thread::sleep_for(
30         std::chrono::milliseconds(100));
31     service.stop();
32 }

```

Результат: deadlock или undefined behavior. Колбэк, переданный в `Invoke`, исполняется внутри `FindMatchingExpectationLocked`, которая уже удерживает `g_gmock_mutex`. Вложенный `EXPECT_CALL` пытается захватить тот же мьютекс повторно. Поскольку `g_gmock_mutex` — это не `std::recursive_mutex`, происходит взаимная блокировка¹⁴.

Даже если бы мьютекс был рекурсивным, такой подход остаётся проблемным: добавление `EXPECT_CALL` после того, как мок уже получал вызовы, является

¹⁴На практике поведение зависит от платформы: на некоторых реализациях `pthread_mutex_t` повторный захват приводит к deadlock, на других — к UB. В любом случае это некорректный код.

undefined behavior по документации GMock¹⁵. Expectation мог бы оказаться в цепочке позже фактического вызова, и GMock не гарантирует корректную верификацию в такой ситуации.

Альтернативный (наивный) подход - поставить expectation не из Invoke, а в теле теста после ожидания завершения фазы:

Listing 10: Запоздалый expectation в теле теста

```
1 // ... service.postConnect(42) + sleep ...
2
3 // Фаза 1 завершена. Пытаемся поставить expectation
4 // по "месту" слишком-- поздно:
5 EXPECT_CALL(observer, onDataReceived(42, "ack:hello"))
6     .Times(1);
7
8 service.postSend(42, "hello");
9 // ...
```

Здесь deadlock не возникает, но результат аналогичен: если onDataReceived вызывается асинхронно рабочим потоком и успевает сработать до регистрации expectation, GMock его не засчитает. Результат: <<Expected: to be called once. Actual: never called.>>

Причина обоих вариантов: GMock проверяет вызовы только по существующим *на момент вызова* expectations. Ни колбэк Invoke, ни тело теста не дают надёжного способа зарегистрировать expectation **между** асинхронными фазами, потому что момент фактического вызова мока контролируется рабочим потоком, а не тестом. Более того, добавление EXPECT_CALL после начала вызовов мока является undefined behavior по документации GMock (см. сноску выше).

4.3. Проблема 3: условное ветвление в runtime

Сценарий: тестируемый код принимает решение в runtime (например, успех или ошибка подключения), и мы хотим строго проверить соответствующую ветку.

Listing 11: Тест с неверным предположением о ветке выполнения

```
1 TEST(RuntimeExpectationsFailing,
2     ConditionalBranch_WrongAssumption_FAILS) {
3     MockObserver observer;
4     MockTransport transport;
5
6     // Транспорт решает вернуть ошибку
```

¹⁵«Important note: EXPECT_CALL must precede the actual function calls. Otherwise the behavior is undefined.» — https://google.github.io/googletest/gmock_for_dummies.html#setting-expectations.

```

7     EXPECT_CALL(transport, connect(1))
8         .WillOnce(Return(false));
9
10    // Мы ожидаем успешный путь:
11    EXPECT_CALL(observer, onConnected(1)).Times(1);
12    EXPECT_CALL(observer, onError(1, _)).Times(0);
13
14    AsyncService service(observer, transport);
15    service.start();
16    service.postConnect(1);
17    std::this_thread::sleep_for(
18        std::chrono::milliseconds(100));
19    service.stop();
20 }

```

Результат: два провала одновременно - `onConnected` «never called» и `onError` «called 1 time, expected 0». Сервис работает корректно (при ошибке подключения вызывает `onError`), но тест падает, потому что `expectations` описывают другую ветку.¹⁶

Причина: GMock не имеет понятия «альтернативных путей». Все *expectations* детерминированы и проверяются при верификации независимо от того, какая ветка кода фактически выполнялась.

4.4. Итоги: ограничения GMock при тестировании runtime-логики

Подведем итог по проблемам, которые удалось продемонстрировать. Все они происходят из одного архитектурного свойства GMock - *expectations статичны и должны быть полностью определены до начала выполнения тестируемого кода*.

1. Невозможность использования runtime-значений в expectations.

Если идентификатор, ключ или иное значение становится известно только в

¹⁶На практике подобные ситуации «глушат» одним из нескольких способов. Способы, которые встречались автору:

- `NiceMock<T>` –подавляет предупреждения о неожиданных вызовах, фактически делая все незадаанные вызовы допустимыми;
- `ON_CALL(...).WillByDefault(...)` –задаёт поведение по умолчанию без какой-либо верификации;
- `EXPECT_CALL(...).WillRepeatedly(...)` –принимает любое количество вызовов с любыми аргументами.

Все три подхода эквивалентны **отсутствию проверок**: тест проходит, но не верифицирует ни аргументы, ни количество, ни порядок вызовов альтернативной ветки.

момент выполнения, `expectation` вынужден использовать матчер `_`. Теряется точность проверки тестируемого кода.

2. Грубая гранулярность сброса `expectations`.

`VerifyAndClearExpectations()`¹⁷ сбрасывает *все* `expectations` объекта. Невозможно обновить *expectation* для одного метода, сохранив остальные - при поэтапном тестировании приходится заново описывать *полное* состояние мока после каждой фазы.

3. Отсутствие условного ветвления `expectations`.

Если тестируемый код выбирает одну из нескольких веток в runtime, тест обязан угадать этот выбор заранее. Описать альтернативные пути и позволить фреймворку проверить фактически выполнившийся - невозможно. Приходится писать различные `workaround`-ы.

В следующем разделе мы предложим расширение, направленное на решение перечисленных проблем, и оценим, какие из них оно устраняет полностью, какие - частично, а какие ограничения сохраняются.

5. Поиски решения

5.1. Мотивация

Как видно из приведённых выше примеров, статическая природа `expectations` накладывает существенные ограничения на тестирование асинхронного кода с условной логикой. Возникает желание устранить эти ограничения на уровне самого фреймворка, а не обходить их очередным `workaround`-ом (костылем).

Концептуально видятся два направления:

1. **Точечное расширение существующего API.** Добавить в GMock механизм, позволяющий `expectation`-у учитывать runtime-состояние, не затрагивая остальную архитектуру.
2. **Замена модели мокирования на конечный автомат (state machine).** Полностью переработать механизм матчинга, заменив линейный список `expectations` графом состояний и переходов.

Именно про эти два подхода дальше и пойдет разговор.

¹⁷https://google.github.io/googletest/gmock_cook_book.html#UsingCheckPoints.

5.2. Предлагаемое решение 1: метод .When()

5.2.1. Требования

Прежде чем переходить к деталям, крайне важно заранее оговорить перечень требований, которые мы бы хотели по возможности соблюсти:

1. **Обратная совместимость.** В кодовой базе любого крупного проекта уже существуют тысячи тестов, написанных на классическом GMock. Расширение не должно менять поведение ни одного существующего теста.
2. **Простота использования.** Решение должно органично вписываться в существующий синтаксис и не ухудшать читаемость тестов. Разработчик, знакомый с GMock, должен понимать семантику расширения.

5.2.2. Идея

Расширение заключается в добавлении одного метода в builder-цепочку EXPECT_CALL(): метод .When(predicate), принимающий предикат `std::function<bool()>`, который вычисляется при каждом вызове мока. Если предикат возвращает `false`, expectation пропускается при матчинге - как будто его не существует.

Ключевой принцип: если .When() не используется, поведение GMock не меняется. Это и обеспечит нам полную обратную совместимость с существующей кодовой базой тестов.

5.2.3. Модификация исходного кода

Изменения затрагивают *ровно* один файл: `gmock-spec-builders.h`.

Шаг 1: добавление поля в ExpectationBase:

Listing 12: Новое поле в ExpectationBase

```
1 class ExpectationBase {
2     protected:
3         // ... существующие поля ...
4
5         // Опциональный runtime предикат-.
6         // Если задан и возвращает false, expectation
7         // пропускается при матчинге.
8         std::function<bool()> runtime_condition_;
9 };
```

Шаг 2: добавление метода When() в TypedExpectation<F>:

Listing 13: Метод When в builder-цепочке

```
1 TypedExpectation& When(std::function<bool()> condition) {
```

```

2     runtime_condition_ = std::move(condition);
3     return *this;
4 }

```

Шаг 3: модификация ShouldHandleArguments:

Listing 14: Модифицированный ShouldHandleArguments

```

1 bool ShouldHandleArguments(
2     const ArgumentTuple& args) const {
3     g_gmock_mutex.AssertHeld();
4     CheckActionCountIfNotDone();
5     return !is_retired()
6         && AllPrerequisitesAreSatisfied()
7         && Matches(args)
8         && EvaluateRuntimeCondition(); // новая функция
9 }
10
11 // Проверяет, что предикат установлен и если это так, то выполняет его
12 bool EvaluateRuntimeCondition() const {
13     return !runtime_condition_ || runtime_condition_();
14 }

```

Такой код *гарантирует*, что если `runtime_condition_` **не** задан, то `EvaluateRuntimeCondition()` возвращает `true` и `ShouldHandleArguments()` работает как раньше.

5.2.4. Проблема верификации и её решение

Модификации только `ShouldHandleArguments()` недостаточно, как раз таки из-за особенностей работы механизма верификации.

Рассмотрим, что происходит при уничтожении mock-объекта: метод `VerifyAndClearExpectationsLocke`¹⁸ проходит по всем expectations и проверяет `IsSatisfied()`. Если expectation с `.When` (`[&]{ return false; }`) имеет `Times(1)`, но `condition` ни разу не вернул `true`, то `call_count_ == 0` и верификация сообщит об ошибке «never called».

Решить эту проблему можно двумя способами.

Способ 1: семантика `AtMost` на стороне теста. Обязать разработчика при написании условных *expectations* использовать `Times(AtMost(1))` вместо `Times(1)`. На первый взгляд это выглядит как ограничение и жуткое неудобство, однако такой подход вводит семантически верную формулировку: при запуске асинхронного сервиса мы не можем знать *точно*, сколько раз будет вызван тот или иной метод и

¹⁸gmock-spec-builders.cc, строка 790: <https://github.com/google/googletest/blob/v1.15.0/googlemock/src/gmock-spec-builders.cc#L790>.

в каком состоянии. Запись `AtMost(N)` честно отражает эту неопределённость. Данный способ не требует никаких дополнительных изменений в коде фреймворка и полностью сохраняет обратную совместимость.

Способ 2: флаг активности на стороне фреймворка. Альтернативный подход — модифицировать сам механизм верификации. В класс `ExpectationBase` добавляется флаг `was_ever_active_`, который устанавливается в `true` при первом вызове `EvaluateRuntimeCondition()`, вернувшем `true`. В методе `VerifyAndClearExpectationsLocked` проверка дополняется условием: если `expectation` имеет `runtime`-предикат, но `was_ever_active_ == false`, он пропускается при верификации. Такой `expectation` никогда не был «виден» матчеру, а значит, предъявлять к нему требования по количеству вызовов некорректно.

Также заметим, что оба способа *совместимы друг с другом* и могут применяться одновременно. В дальнейшем изложении мы рассмотрим реализацию второго способа, поскольку он снимает с разработчика необходимость помнить о семантических нюансах `Times()` при каждом использовании `.When()`.

Listing 15: Способ 2: модификация верификации с учётом *runtime condition*

```
1 // В VerifyAndClearExpectationsLocked:
2 for (auto it = untyped_expectations_.begin();
3     it != untyped_expectations_.end(); ++it) {
4     ExpectationBase* const exp = it->get();
5     if (exp->IsOverSaturated()) {
6         expectations_met = false;
7     } else if (!exp->IsSatisfied()) {
8         // НОВОЕ: expectation с runtime_condition_,
9         // который ни разу не был активен,
10        // считается удовлетворённым.
11        if (exp->HasRuntimeCondition()
12            && !exp->WasEverActive()) {
13            continue; // пропускаем верификацию
14        }
15        expectations_met = false;
16        // ... формирование ошибки ...
17    }
18 }
```

5.2.5. Решение проблемы 3 с помощью `.When()`

Каждую из приведённых выше проблем предложенный подход с `.When()` позволяет решить. Однако, чтобы не перегружать изложение техническими деталями,

рассмотрим применение только к одному, по-моему мнению , самому важному и показательному сценарию 3.

Напомню в чем была его суть: тестируемый код принимает решение в *runtime* (например, успех или ошибка подключения), и мы хотим строго проверить соответствующую ветку.

Listing 16: Тест с условным ветвлением через .When()

```
1  TEST(RuntimeExpectationsFixed,
2      ConditionalBranch_WithWhen) {
3      MockObserver observer;
4      MockTransport transport;
5
6      std::atomic<bool> connect_succeeded{false};
7      // Путь A: успешное подключение
8      EXPECT_CALL(observer, onConnected(1))
9          .When([&] {
10              return connect_succeeded.load();
11          })
12          .Times(1);
13
14      // Путь B: ошибка подключения
15      EXPECT_CALL(observer, onError(1, _))
16          .When([&] {
17              return !connect_succeeded.load();
18          })
19          .Times(1);
20
21      AsyncService service(observer, transport);
22      service.start();
23      service.postConnect(1);
24      std::this_thread::sleep_for(
25          std::chrono::milliseconds(100));
26      service.stop();
27 }
```

Механизм работы:

1. Worker thread вызывает `transport.connect(1)`. Результат вызова заранее неизвестен тесту- он определяется реализацией транспорта (или его мока, настроенного извне). По результату обновляется флаг. `connect_succeeded`.
2. В зависимости от возвращённого значения `AsyncService` вызывает либо `observer.onConnected(1)`, либо `observer.onError(1, "connect failed")`.

3. GMock входит в `FindMatchingExpectationLocked()` и обходит вектор `expectations` с конца. Допустим, подключение завершилось ошибкой (`connect_succeeded == false`):

- expectation на `onError` с предикатом `When([ &] { return !connect_succeeded.load(); })` - condition возвращает `true`, аргументы совпадают - **expectation найден**;
- expectation на `onConnected` с предикатом `When([ &] { return connect_succeeded.load(); })` - condition возвращает `false`, expectation **пропускается** матчером.

Для успешного подключения ситуация будет зеркальной.

4. При уничтожении мока:

- `onConnected - was_ever_active_ == false`, верификация пропускается;
- `onError - Times(1), call_count_ == 1` - удовлетворён.

5. Как результат, тест проходит при обоих сценариях.

5.2.6. Совместимость с существующим API

Расширение совместимо со всеми существующими конструкциями GMock:

- **Times, WillOnce, WillRepeatedly**: работают как прежде. `.When()` добавляет дополнительное условие в конъюнкцию, не заменяя существующие.
- **InSequence / .After()**: `AllPrerequisitesAreSatisfied()` и `EvaluateRuntimeCondition()` объединяются через AND. Порядок проверки: сначала `prerequisites`, потом `condition`. Если `prerequisites` не удовлетворены, `condition` не вычисляется, из-за ленивости конъюнкции в C++.
- **RetiresOnSaturation**¹⁹: *retire* происходит при насыщении expectation, `.When()` не влияет на логику механизма *retire*.
- **NiceMock / StrictMock**²⁰: если ни один *expectation* не подошёл (все conditions `false`, или нет matchers), поведение определяется режимом мока — предупреждение или assertion failure, как и раньше.
- **Тесты без .When()**: `runtime_condition_` по умолчанию пуст, `EvaluateRuntimeCondition()` возвращает `true`. Таким образом новая логика **не** повлияет на выполнение уже написанных тестов, так как системный стек вызовов будет идентичным (с точностью до проверки того, что предикат не выставлен).

¹⁹https://google.github.io/googletest/gmock_cheat_sheet.html#StickyExpectations.

²⁰https://google.github.io/googletest/gmock_cook_book.html#NiceMocksStrictMocksAndNaggyMocks.

5.2.7. Плюсы такого подхода

Резюмируя, обозначим конкретные плюсы такого подхода к изменениям:

1. **Полная обратная совместимость.** Если `.When()` не используется, то *Все существующие тесты продолжают работать без каких-либо изменений.*
2. **Минимальный объём модификаций.** Расширение затрагивает только два места в коде фреймворка.
Ни один из существующих публичных интерфейсов GMock не изменяется.
3. **Естественный синтаксис.** Метод `.When()` органично встраивается в существующую декларативную build-цепочку фреймворка.
Разработчик, знакомый с GMock, может читать и писать такие тесты без изучения дополнительной документации.
4. **Декларативность вместо императивности.** Вместо ручной синхронизации через какие-то `workaround` и повторной расстановки `expectations` между каждой фазой теста,
*от разработчика требуется описать все **ожидаемые** ветки поведения кода заранее, а фреймворк сам выберет необходимый сценарий на основе runtime состояния.*
5. **Тест не предсказывает, а описывает.** Ключевое концептуальное изменение: *тест больше не обязан заранее знать, какой именно путь выберет тестируемый код.*

5.3. Предлагаемое решение 2: State Machine

Расширение `.When()` представляет собой точечное, минимально решение, которое можно внедрить в существующую кодовую базу без значительных затрат. Однако оно не устраняет *корневую причину* описанных проблем - архитектуру GMock, построенную на иммутабельных *expectations* и плоском списке ожиданий без понятия состояния.

В этом разделе мы рассмотрим концептуально иной подход: построение тестов на основе *конечного автомата*, в котором поведение мок-объектов определяется текущим состоянием системы.

5.3.1. Мотивация

Асинхронный сервис по своей природе является конечным автоматом: он переходит между состояниями (инициализация, подключение, передача данных, ошибка) в зависимости от внешних событий. Однако GMock не предоставляет

средств для описания этой модели - разработчик вынужден «сплющивать» все возможные ожидания в один плоский список, теряя информацию о состояниях и переходах между ними.

Идея *State Machine* заключается в том, чтобы восстановить эту структуру: описать состояния тестируемой системы, ожидания в каждом из них и условия переходов. Таким образом тест становится *моделью* тестируемого кода, а не набором изолированных expectations.

5.3.2. Модель

Предлагаемое решение основано на следующих сущностях:

1. **StateMachine** – центральный объект, хранящий граф состояний, текущую позицию (курсор) и глобальное хранилище переменных (Store).
2. **Node (нода)** – состояние системы. Содержит набор expectations, которые *активируются* при входе в ноду и *деактивируются* при выходе. При повторном входе в ноду счётчики вызовов *продолжаются*, а не сбрасываются.
3. **Transition (ребро)** – переход между нодами. Привязан к вызову конкретного метода; переход происходит в зависимости от результата этого вызова. Метод может быть как замокированным, так и реальным (через делегирование к настоящей реализации).
4. **Store** – хранилище глобального состояния автомата. Переменные регистрируются через `AddState(key, initial_value)` и доступны из expectations, рёбер и тела теста через `GetState<T>(key)` / `SetState(key, value)`.

Обработка вызова.

При каждом вызове замокированного метода движок выполняет следующую последовательность действий:

1. Определяется текущая нода.
2. Проверяется, существует ли исходящее ребро из текущей ноды, соответствующее данному вызову и его результату.
3. Если ребро найдено – выполняется action ребра, курсор перемещается в целевую ноду. Expectations старой ноды деактивируются, expectations новой – активируются.
4. Если ребро не найдено – вызов обрабатывается expectations текущей ноды (семантика self-loop).
5. Если ни ребро, ни expectation не найдены – фиксируется ошибка (аналог «unexpected call» в GMock).

Верификация.

При вызове `sm.Verify()` проверяются expectations всех *посещённых* нод. Ноды, в которые автомат не входил, не верифицируются – аналогично тому, как неактивированные expectations с `.When()` пропускаются при проверке. Верификация может быть вызвана в любой момент теста, а не только в конце.

5.3.3. API-скетч

Рассмотрим применение предлагаемого подхода к сценарию `NetworkMonitor` – тому самому, который использовался при описании проблем в разделе ??.

Listing 17: Тест с использованием State Machine

```
1  TEST(NetworkMonitor, DynamicConnectScenario) {
2      MockTransport transport;
3      MockObserver observer;
4
5      StateMachine sm;
6
7      // Глобальное состояние автомата
8      sm.AddState("connection_id", -1);
9
10     // Ноды
11     auto& init = sm.AddNode("Init");
12     auto& connecting = sm.AddNode("Connecting");
13     auto& connected = sm.AddNode("Connected");
14     auto& error = sm.AddNode("ConnectionError");
15
16     // Init: настраиваем параметры подключения
17     init.Expect(transport, getHost())
18         .WillRepeatedly(Return("192.168.1.1"));
19     init.Expect(transport, getPort())
20         .WillRepeatedly(Return(8080));
21
22     // Connected: сохраняем id, проверяем отправку
23     connected.Expect(observer, onConnected(_))
24         .Times(1)
25         .WillOnce([&](int id) {
26             sm.SetState("connection_id", id);
27         });
28     connected.Expect(transport, send(_, _))
29         .Matching([&](int id, auto) {
30             return id ==
31                 sm.GetState<int>("connection_id");
```

```

32     })
33     .Times(AtLeast(1))
34     .WillRepeatedly(Return(true));
35
36     // Error: ничего не отправляем
37     error.Expect(transport, send(_, _)).Times(0);
38     error.Expect(observer, onError(_)).Times(1);
39
40     // Рёбра: реальный connect определяет путь
41     sm.Transition(init, connecting)
42         .OnCall(transport, connect())
43         .WillOnce(InvokeReal(&transport));
44
45     sm.Transition(connecting, connected)
46         .WhenResult(transport, connect(),
47                     Eq(true));
48
49     sm.Transition(connecting, error)
50         .WhenResult(transport, connect(),
51                     Eq(false));
52
53     // Запуск
54     sm.Start(init);
55     NetworkMonitor monitor(&transport, &observer);
56     monitor.run();
57
58     // Проверка из тела теста
59     EXPECT_NE(
60         sm.GetState<int>("connection_id"), -1);
61
62     // Верификация всех посещённых нод
63     sm.Verify();
64 }

```

5.3.4. Решение исходных проблем

Рассмотрим, как данная модель соотносится с тремя проблемами, описанными в разделе ??.

Проблема 1: замороженные matchers.

Matchers в expectations нод не замораживаются при объявлении. Метод `Matching()` принимает предикат, который вычисляется при каждом вызове и обращается

к Store за актуальным значением. В примере выше `matcher` на `send(_, _)` читает `sm.GetState<int>("connection_id")` в момент вызова, а не в момент объявления.

Проблема 2: грубая гранулярность сброса.

`VerifyAndClearExpectations` не используется. Фазовость является естественным следствием переходов между нодами: при выходе из ноды её `expectations` деактивируются, при входе в следующую – активируются новые. Каждая нода – это фаза, и переход между фазами не требует ручного сброса.

Проблема 3: условное ветвление.

Ветвление описано непосредственно в графе автомата: из ноды `Connecting` ведут два ребра – в `Connected` и в `ConnectionError`, выбор между которыми определяется результатом реального вызова `connect()`. Тест не предсказывает результат – он описывает оба сценария, а автомат выбирает нужный путь в runtime.

5.3.5. Совместимость с GTest

Предлагаемый подход *не является расширением GMock* — это самостоятельный движок мокирования, использующий GTest исключительно как test runner и механизм репорта ошибок (`ADD_FAILURE()`, `EXPECT_*`, `ASSERT_*`). Библиотека `matchers` GMock (`Eq`, `_`, `Gt`, `HasSubstr` и т. д.) может быть переиспользована, поскольку `matchers` реализованы независимо от инфраструктуры `EXPECT_CALL` и не зависят от `g_gmock_mutex`.

Потокобезопасность обеспечивается на уровне `StateMachine`: переходы между нодами и доступ к Store защищаются единым мьютексом, а `expectations` конкретной ноды иммутабельны после объявления и обрабатываются только пока нода активна.

5.3.6. Ограничения и направления дальнейшего исследования

Предложенная модель носит *концептуальный характер* и требует дальнейшей проработки:

- **Реализация.** Представленный API является скетчем. Полноценная реализация потребует значительных трудозатрат, включая разработку `dispatch`-механизма, интеграцию с GTest и обширное тестирование на реальных сценариях.
- **Порог входа.** Описание тестов в виде конечного автомата сложнее привычного линейного списка `EXPECT_CALL`. Для простых синхронных сценариев такой подход избыточен.

- **Compile-time рефлексия.** Появление compile-time рефлексии в стандарте C++26 потенциально позволит автоматизировать генерацию dispatch-таблиц и устранить необходимость явного перечисления методов при описании рёбер автомата.

В отличие от `.When()`, которое представляет собой прагматичную заплатку, применимую здесь и сейчас, State Machine — это направление для построения тестовой инфраструктуры, в которой асинхронность является не исключением, а нормой.

6. Сравнительный анализ альтернативных фреймворков

6.1. Критерии сравнения

Для объективного сравнения фреймворков мокирования в контексте проблем *runtime expectations* сформулируем два ключевых критерия:

1. **Решение технических проблем (П1–П3).** Устраняет ли фреймворк на архитектурном уровне продемонстрированные проблемы: фиксацию матчеров по значению (П1), грубую гранулярность сброса expectations (П2) и невозможность условного ветвления (П3)?

Решение должно быть средствами самого фреймворка, а не сторонними workaround-ами.

2. **Порог входа и объём переобучения.** Сколько новых концепций, макросов и паттернов нужно освоить разработчику, знакомому с GMock? Насколько синтаксис нового фреймворка отличается от привычного `EXPECT_CALL(...).WillOnce(...)`?

Этот критерий особенно важен для крупных проектов, где переход на другой фреймворк затрагивает сотни тестов и десятки разработчиков.

6.2. Trompeloeil

6.2.1. Общие сведения

Trompeloeil²¹ — header-only фреймворк мокирования для C++14+, разрабатываемый с 2014 года. Распространяется под Boost Software License. Потребно-безопасен. Может использоваться совместно с любым тестовым раннером (GTest, Catch2, doctest и др.) через специализированные адаптеры.

²¹Репозиторий: <https://github.com/rollbear/trompeloeil>. Документация: <https://trompeloeil.github.io/trompeloeil/reference.html>.

Принципиальное архитектурное отличие от *GMock* - это *expectations*, которые являются полноценными C++-объектами с *RAII*-семантикой.

6.2.2. Архитектура

Мок-метод генерируется макросом `MAKE_MOCK`²² и при вызове передаёт управление функции `mock_func()`, которая:

1. Захватывает глобальный мьютекс.
2. Вызывает `find(active_list, params)`- линейный обход интрузивного списка `expectations`.
3. Для каждого `expectation` проверяет: `match_parameters(val, params) && match_conditions(params)`.
4. При совпадении- выполняет `side-effects`, инкрементирует счётчик, возвращает значение. Если `expectation` «насытился» - переносит его из `active` в `saturated`.
5. Если совпадение не найдено, то тест падает.

Механизм верификации располагается в деструкторе `call_matcher`²³: если `expectation` не был удовлетворён к моменту уничтожения объекта, то фреймворк сообщает об ошибке. Это означает, что верификация привязана к самому `expectation`-у, а не к мок-объекту.

6.2.3. Ключевые отличия синтаксиса

Синтаксис двух фреймворков довольно похож. Видно, что фреймворк вдохновлялся семантикой *GMock*.

Listing 18: Сравнение синтаксиса: *GMock* vs *Trompeloeil*

```
1 // GMock:
2 EXPECT_CALL(mock, foo(42))
3     .Times(1)
4     .WillOnce(Return(true));
5
6 // Trompeloeil:
7 REQUIRE_CALL(mock, foo(42))
8     .RETURN(true);
9 // (Times(1) -- по умолчанию для REQUIRE_CALL)
```

²²Исходный код: `trompeloeil.hpp` — весь фреймворк сосредоточен в одном заголовочном файле, см. <https://github.com/rollbear/trompeloeil/blob/main/include/trompeloeil.hpp>.

²³Деструктор `call_matcher` вызывает `report_missed()`, если `expectation` не был удовлетворён. См. документацию: https://trompeloeil.github.io/trompeloeil/reference.html#REQUIRE_CALL.

Приведём подробное сопоставление API двух фреймворков.

Макросы объявления expectations.

GMock	Trompeloeil	Семантика
EXPECT_CALL(...) .Times(1)	REQUIRE_CALL()	Ровно 1 вызов; верификация при выходе из скоупа
EXPECT_CALL(...) .Times(AnyNumber())	ALLOW_CALL()	0...∞ вызовов
EXPECT_CALL(...) .Times(0)	FORBID_CALL()	Запрет вызова
ON_CALL(...) .WillByDefault(...)	ALLOW_CALL()	Поведение по умолчанию
—	NAMED_REQUIRE_CALL() NAMED_ALLOW_CALL() NAMED_FORBID_CALL()	Возвращает <code>unique_ptr<expectation></code> для ручного управления lifetime

Модификаторы (clauses) expectations.

GMock	Trompeloeil	Примечание
.Times(n) .Times(m, n)	.TIMES(n) .TIMES(m, n)	Диапазон вызовов. Аргументы должны быть <code>constexpr</code>
—	.RT_TIMES(n) .RT_TIMES(m, n)	Runtime-значения (не <code>constexpr</code>); аналога в GMock нет
.Times(AtLeast(n))	.TIMES(AT_LEAST(n))	$n \dots \infty$
.Times(AtMost(n))	.TIMES(AT_MOST(n))	$0 \dots n$
.WillOnce(Return(v))	.RETURN(v)	Возвращаемое значение (захват по значению)
—	.LR_RETURN(v)	Возвращаемое значение (захват по ссылке)
.WillOnce(Invoke(f))	.SIDE_EFFECT(expr)	Побочный эффект (захват по значению)
—	.LR_SIDE_EFFECT(expr)	Побочный эффект (захват по ссылке)
.WillOnce(Throw(e))	.THROW(e) / .LR_THROW(e)	Выброс исключения
.InSequence(seq) .After(e)	.IN_SEQUENCE(seq...)	Упорядочение вызовов
SaveArg<N>(&var)	.LR_SIDE_EFFECT	Захват аргумента в переменную

Отдельно стоит отметить различие в захвате переменных: в GMock все матчеры захватывают значения, а действия (actions) работают с произвольными типами. В Trompeloeil каждый модификатор существует в двух вариантах: без префикса (захват по значению, аналогично GMock) и с префиксом LR_ (*local reference* – захват по ссылке). Это явное разделение снижает вероятность ошибки, связанной

c dangling reference.

Упорядочение. В GMock доступны два механизма: класс `InSequence` и метод `.After()` (явная зависимость между expectations). В Trompeloeil используется один механизм - объект `trompeloeil::sequence`, передаваемый в `.IN_SEQUENCE(seq...)`. Один expectation может участвовать в нескольких параллельных последовательностях одновременно. Прямого аналога `.After()` нет, но `IN_SEQUENCE` покрывает те же сценарии.

Важная деталь: `.IN_SEQUENCE()` в сочетании с `.TIMES()` работает «жадно» - фреймворк остаётся на текущем expectation, пока тот может принимать вызовы (до верхней границы `TIMES`), и переходит к следующему только по насыщению или при совпадении с другим expectation.

NAMED-варианты и управление lifetime.

Макросы `NAMED_REQUIRE_CALL()`, `NAMED_ALLOW_CALL()` и `NAMED_FORBID_CALL` возвращают `std::unique_ptr<trompeloeil::expectation>`. С этим объектом можно:

- **Перемещать** (`std::move`) - хранить в контейнерах, передавать между функциями, размещать в test-fixture.
- **Запрашивать статус:** `is_satisfied()` (нижняя граница `TIMES` достигнута) и `is_saturated()` (верхняя граница достигнута).
- **Уничтожать досрочно:** `ptr.reset()` или выход из скоупа - при этом происходит верификация (аналог `VerifyAndClearExpectations`, но для одного expectation).
- **Возможность модификации:** возможность получить указатель, позволяет при необходимости достаточно просто и читаемо модифицировать фреймворк под нужды программиста.

Копирование невозможно, так как это `std::unique_ptr`.

Клонирования и «перезапуска» expectation нет - для нового expectation на тот же метод нужно создать новый `NAMED_*` объект.

6.2.4. Проверка проблемы П1: runtime-значения

В GMock тест `DynamicId_ExpectationFrozenAtSetup` падает, потому что `EXPECT_CALL` копирует значение переменной на момент объявления. В Trompeloeil проблема решается RAII-скоупами:

Listing 19: Trompeloeil: expectation с runtime-значением (П1)

```
1 TEST(Trompeloeil, DynamicId_ExpectationFrozenAtSetup) {
2     MockObserver observer;
```

```

3 MockTransport transport;
4 int captured_id = -1;
5
6 // Фаза 1: узнаём session_id
7 {
8     REQUIRE_CALL(transport, connect(_))
9         .RETURN(true);
10    REQUIRE_CALL(observer, onConnected(_))
11        .LR_SIDE_EFFECT(captured_id = _1);
12
13    AsyncService service(observer, transport);
14    service.start();
15    service.postConnect(42);
16    std::this_thread::sleep_for(
17        std::chrono::milliseconds(100));
18    service.stop();
19 } // expectations верифицированы, captured_id == 42
20
21 // Фаза 2: используем captured_id в НОВОМ expectation
22 {
23     REQUIRE_CALL(transport,
24         send(captured_id, "hello"))
25         .RETURN(true);
26     REQUIRE_CALL(observer,
27         onDataReceived(captured_id, "ack:hello"));
28
29     AsyncService service(observer, transport);
30     service.start();
31     service.postSend(captured_id, "hello");
32     std::this_thread::sleep_for(
33         std::chrono::milliseconds(100));
34     service.stop();
35 }
36 }

```

Expectation во второй фазе создаётся *после* того, как captured_id получил реальное значение. Проблема P1 **решена**.

Замечание. Аналогичный двухфазный подход возможен и в GMock: достаточно дождаться завершения первой фазы (когда SaveArg запишет реальное значение в captured_id), а затем объявить новый EXPECT_CALL с уже корректным значением. Однако такой подход работает только для простых линейных сценариев. При наличии нескольких параллельных expectations с разным временем жизни GMock не

предоставляет механизма гранулярного управления - это Проблема 2 (П2), которую RAII-скоупы Trompeloeil решают естественным образом.

6.2.5. Проверка проблемы П2: гранулярность сброса

Listing 20: Trompeloeil: гранулярный контроль expectations (П2)

```
1 TEST(Trompeloeil, ScopedExpectations_P2) {
2     MockObserver observer;
3     MockTransport transport;
4
5     // Фоновый expectation на весь тест
6     FORBID_CALL(observer, onError(_, _));
7
8     // Фаза 1: connect
9     {
10         REQUIRE_CALL(transport, connect(42))
11             .RETURN(true);
12         REQUIRE_CALL(observer, onConnected(42));
13
14         AsyncService service(observer, transport);
15         service.start();
16         service.postConnect(42);
17         std::this_thread::sleep_for(
18             std::chrono::milliseconds(100));
19         service.stop();
20     }
21     // Expectations фазы 1 уничтожены и верифицированы.
22     // FORBID_CALL(onError) попрежнему- активен!
23     //
24     // КЛЮЧЕВОЙ МОМЕНТ: если здесь вызвать
25     // transport.connect(42), тест УПАДЁТ с ошибкой
26     // "No match for call", потому что REQUIRE_CALL
27     // на connect уже уничтожен при выходе из {}.
28     // При этом FORBID_CALL на onError остаётся.
29
30     // Фаза 2: send с( НОВЫМ connect)
31     {
32         // Новый expectation на connect -- с ДРУГИМ
33         // поведением, чем в фазе 1:
34         REQUIRE_CALL(transport, connect(42))
35             .RETURN(false); // теперь возвращаем false
36         REQUIRE_CALL(transport, send(42, "hello"))
```

```

37         .RETURN(true);
38     REQUIRE_CALL(observer,
39         onDataReceived(42, "ack:hello"));
40
41     AsyncService service(observer, transport);
42     service.start();
43     // connect(42) теперь вернёт false --
44     // это ДРУГОЙ expectation, не из фазы 1.
45     service.postConnect(42);
46     std::this_thread::sleep_for(
47         std::chrono::milliseconds(50));
48     service.postSend(42, "hello");
49     std::this_thread::sleep_for(
50         std::chrono::milliseconds(100));
51     service.stop();
52 }
53 // FORBID_CALL(onError) попрежнему- активен
54 // и был бы нарушен, если бы onError вызвался.
55 }

```

Ключевое отличие от GMock: фазовые expectations уничтожаются при выходе из {}, что доказывается тем, что во второй фазе мы объявляем *новый* REQUIRE_CALL (transport, connect(42)) с *другим* поведением (RETURN(false) вместо RETURN(true)). Если бы expectation из фазы 1 оставался активен, возникла бы коллизия. При этом «фоновый» FORBID_CALL живёт до конца теста — попытка вызвать onError в любой фазе приведёт к падению.

В GMock для аналогичного сценария потребовался бы вызов VerifyAndClearExpectations (&transport), который сбросил бы *все* expectations транспорта, включая те, которые мы хотели бы сохранить.

Как итог - П2 решена.

6.2.6. Проверка проблемы П3: условное ветвление

Для корректного сравнения сначала покажем тест, эквивалентный GMock-версии — с теми же строгими ожиданиями:

Listing 21: Trompeloeil: эквивалент GMock-теста (П3) — строгие ожидания

```

1 TEST(Trompeloeil,
2     ConditionalBranch_Strict_FAILS) {
3     MockObserver observer;
4     MockTransport transport;
5
6     REQUIRE_CALL(transport, connect(1))

```

```

7         .RETURN(false);
8
9         // Строгие ожидания: ожидаем onConnected,
10        // запрещаем onError
11        REQUIRE_CALL(observer, onConnected(1));
12        FORBID_CALL(observer, onError(1, _));
13
14        AsyncService service(observer, transport);
15        service.start();
16        service.postConnect(1);
17        std::this_thread::sleep_for(
18            std::chrono::milliseconds(100));
19        service.stop();
20
21        // Результат: тест ПАДАЕТ с двумя ошибками ---
22        // onConnected "never called" и
23        // onError нарушил FORBID_CALL.
24        // Ситуация идентична GMock версии-.
25    }

```

Как видно, при использовании строгих ожиданий (`REQUIRE_CALL()` + `FORBID_CALL()`) Trompeloeil ведёт себя *идентично GMock*: тест падает, потому что *expectations* описывают не ту ветку.

Единственный способ не падать - это заменить строгие ожидания на «разрешающие» (`ALLOW_CALL()`) :

Listing 22: Trompeloeil: ослабление ожиданий через `ALLOW_CALL()`

```

1 TEST(Trompeloeil,
2     ConditionalBranch_AllowBothPaths) {
3     MockObserver observer;
4     MockTransport transport;
5
6     REQUIRE_CALL(transport, connect(1))
7         .RETURN(false);
8
9     // Ослабляем: ALLOW_CALL допускает 0..inf вызовов
10    ALLOW_CALL(observer, onConnected(1));
11    ALLOW_CALL(observer, onError(1, _));
12
13    AsyncService service(observer, transport);
14    service.start();
15    service.postConnect(1);

```

```

16     std::this_thread::sleep_for(
17         std::chrono::milliseconds(100));
18     service.stop();
19
20     // Тест проходит, но ALLOW_CALL допускает
21     // и полное отсутствие вызова.
22     // Невозможно ТРЕБОВАТЬ, чтобы при connect==false
23     // был вызван именно onError.
24 }

```

Тест не падает, *но* ценой потери строгости: `ALLOW_CALL()` допускает и полное отсутствие вызова, то есть тест пройдёт даже если ни `onConnected()`, ни `onError()` не будут вызваны вовсе. Единственный workaround - это ручной счётчик в `LR_SIDE_EFFECT()` с последующей проверкой через `EXPECT_EQ()`, что выводит верификацию за пределы фреймворка. Проблема ПЗ не решена.

6.2.7. Возможность реализации `.When()` в *Trompeloeil*

Так как ПЗ не решается в рамках рассматриваемого аналога, то логично задать вопросом: «а можно ли реализовать, что-то на подобии ранее описанного метода `.When()`?»

И ответом на этот вопрос будет уверенное «Да».

Так как архитектура *Trompeloeil* существенно упрощает внедрение аналога `.When()`. Метод матчинга `call_matcher::matches()` является обычным виртуальным методом, а верификация - частью деструктора одного класса. Для реализации достаточно:

1. Добавить поле `std::function<bool()> when_condition_` в `call_matcher`.
2. В `matches()` добавить проверку: `if (when_condition_ && !when_condition_()) return false;`
3. В деструкторе пропускать верификацию для expectations, у которых предикат ни разу не вернул `true`.

В сравнении с `GMock`, где модификация затрагивает шаблонный `FunctionMocker` и глобальный метод `VerifyAndClearExpectationsLocked`, в *Trompeloeil* изменения локализованы в одном классе - `call_matcher`. Объём модификации - порядка 15–20 строк.

6.2.8. Оценка по критериям

Критерий 1: решение технических проблем. П1 и П2 решены на архитектурном уровне благодаря RAII-семантике expectations. ПЗ не решена - отсутствует

механизм условной активации expectation на основе внешнего состояния. Однако реализация аналога `.When()` в Trompeloeil проще, чем в GMock.

Критерий 2: порог входа. Основные концепции (expectations, матчеры, side-effects, return values) очень схожи с GMock. Новые понятия:

- RAII-скоупы вместо верификации при уничтожении мока
- NAMED_* варианты для ручного управления lifetime.
- Отсутствие InSequence как класса - вместо него объект `trompeloeil::sequence`.

То есть для понимания основной концепции работы с данным тестовым фреймворком достаточно иметь базовое представление об RAII.

Синтаксис близок к GMock, переобучение можно оценить как **минимальное**: 1-2 дня для разработчика, знакомого с GMock и C++.

6.3. FakeIt

FakeIt –header-only фреймворк мокирования для C++11, разработанный Evan Re’er (лицензия MIT). Принципиальное отличие от GMock и Trompeloeil — **не требует макросов для объявления мок-классов**. Мок создаётся из интерфейса одной строкой:

```
1 Mock<ITransport> mock;  
2 ITransport& obj = mock.get();
```

Достигается это через прямую манипуляцию vtable на уровне бинарного кода.

6.3.1. Архитектура

Фундамент FakeIt - класс VirtualTableBase (файл `mockutils/gcc/VirtualTable.hpp`). При создании `Mock<C>` фреймворк:

1. Выделяет массив указателей `void**` размером с vtable класса `c`.
2. Инициализирует все слоты указателем на метод `unmocked()` - «заглушку», которая бросает исключение при вызове немокированного метода.
3. Подменяет `vp_ptr` (первое поле объекта) на свою vtable.
4. Использует `union_cast` для преобразования указатель-на-метод в индекс vtable.

Когда метод мокируется через `When(Method(mock, foo))`, соответствующий слот vtable начинает указывать на `RecordedMethodBody<R, arglist...>`, который содержит:

- `_invocationHandlers`- вектор пар (`matcher`, `handler`), обходимый **с конца** (LIFO): последний зарегистрированный стаб имеет приоритет.
- `_actualInvocations` - вектор записанных вызовов, используемый при верификации.

При вызове мок-метода `handleMethodInvocation()`:

1. Создает `ActualInvocation` с порядковым номером и аргументами.
2. Ищет подходящий `handler` через обход с конца (LIFO)²⁴.
3. Если найден - записывает вызов, выполняет `handler` (`Return/Do/Throw`).
4. Если не найден - бросает `UnexpectedMethodCallException`.

Ключевая парадигма `FakeIt` - **Arrange-Act-Assert** (`stub-then-verify`), в отличие от `record-then-verify` у `GMock`. `Stubbing` и верификация –разделённые фазы:

- `When(...)` - задаёт поведение (что возвращать при вызове).
- `Verify(...)` –проверяет постфактум, был ли вызов (сколько раз, с какими аргументами).

Можно стабить без верификации и верифицировать без стабов. Это принципиально отличает `FakeIt` от `GMock`, где `expectation` объединяет матчер, действие и ограничение на количество вызовов в один объект.

Дополнительно `FakeIt` - единственный из рассматриваемых фреймворков с поддержкой **spying**: перехват вызовов с сохранением оригинальной реализации.

6.3.2. Синтаксис: сравнение с `GMock`

Приведём подробное сопоставление API двух фреймворков.

Создание мока и объявление поведения.

²⁴Обход с конца - общий паттерн для всех рассматриваемых фреймворков (`GMock`, `Trompeloeil`, `FakeIt`). Причина: `expectations` описываются от общего к частному - сначала широкое поведение по умолчанию, затем более специфичные уточнения. *LIFO*-порядок гарантирует, что последнее ожидание проверяется первым и перехватывает вызов, не требуя модификации или удаления ранее объявленных `expectations`.

GMock	FakeIt	Семантика
<pre>class MockT : public IT { MOCK_METHOD(...) }; MockT mock;</pre>	<pre>Mock<IT> mock; IT& obj = mock.get();</pre>	Создание мока. FakeIt не требует ручного объявления класса
<pre>EXPECT_CALL(mock, foo(42)) .WillOnce(Return(true));</pre>	<pre>When(Method(mock, foo) .Using(42)) .Return(true);</pre>	Стаб + expectation (GMock) vs только стаб (FakeIt)
<pre>ON_CALL(mock, foo(_)) .WillByDefault(Return(true));</pre>	<pre>When(Method(mock, foo)) .AlwaysReturn(true);</pre>	Поведение по умолчанию
—	<pre>Fake(Method(mock, foo));</pre>	Стаб без поведения (void) или с дефолтным значением
Верификация в де-структоре	<pre>Verify(Method(mock, foo) .Using(42)).Exactly(1);</pre>	Постфактум-верификация

Модификаторы стабирирования и верификации.

GMock	FakeIt	Примечание
<code>.WillOnce(Return(v))</code>	<code>.Return(v)</code>	Однократное возвращаемое значение
<code>.WillRepeatedly(Return(v))</code>	<code>.AlwaysReturn(v)</code>	Возвращаемое значение на все вызовы
<code>.WillOnce(Invoke(f))</code>	<code>.Do(f)</code>	Однократное действие (лямбда)
<code>.WillRepeatedly(Invoke(f))</code>	<code>.AlwaysDo(f)</code>	Действие на все вызовы
<code>.WillOnce(Throw(e))</code>	<code>.Throw(e)</code>	Выброс исключения
<code>Eq(42) / _</code>	<code>.Using(42) / _</code>	Матчинг аргументов
<code>.Times(n)</code>	<code>.Exactly(n)</code>	Ровно <i>n</i> вызовов (только при Verify)
<code>.Times(AtLeast(n))</code>	<code>.AtLeast(n)</code>	Не менее <i>n</i> (только при Verify)
<code>.Times(AtMost(n))</code>	<code>.AtMost(n)</code>	Не более <i>n</i> (только при Verify)
<code>.Times(0)</code>	<code>.Never()</code>	Запрет вызова (только при Verify)
<code>SaveArg<0>(&var)</code>	<code>.Do([&](int x){ var = x; })</code>	Захват аргумента
<code>.InSequence(seq) .After(e)</code>	<code>Verify(seq1 + seq2)</code>	Верификация порядка (только постфактум)

Управление состоянием мока.

GMock	FakeIt	Семантика
VerifyAndClearExpectations(&mock)	mock.ClearInvocationHistory()	GMock: сброс всего. FakeIt: сброс только записанных вызовов, стабы остаются
—	mock.Reset()	Полный сброс: стабы + вызовы
NiceMock<T>	Fake(Method(...))	Подавление ошибок на нестабильные методы
StrictMock<T>	По умолчанию	FakeIt бросает исключение на нестабильный вызов
—	Spy(Method(mock, foo))	Перехват с сохранением оригинальной реализации

Ключевое архитектурное отличие: в GMock `EXPECT_CALL` объединяет матчер, действие и ограничение на количество вызовов в одном объекте. В FakeIt стабирирование (`when`) и верификация (`verify`) — это два полностью независимых механизма. Это даёт гибкость (можно стабить без верификации и наоборот), но лишает возможности ловить ошибки «на лету» — все проверки происходят только постфактум.

6.3.3. Проверка проблемы П1: runtime-значения в expectations

Listing 23: FakeIt: runtime-значения в expectations (П1)

```

1  TEST(FakeIt, DynamicId_ExpectationFrozenAtSetup) {
2      Mock<IEventObserver> mockObserver;
3      Mock<ITransport> mockTransport;
4      int captured_id = -1;
5
6      // Фаза 1: стабим с любыми аргументами,
7      // захватываем id через лямбду
8      When(Method(mockTransport, connect))
9          .AlwaysDo([] (int) { return true; });
10     When(Method(mockObserver, onConnected))
11         .AlwaysDo([&] (int id) {
12             captured_id = id;
13         });
14
15     {
16         AsyncService service(mockObserver.get(),
17                               mockTransport.get());
18         service.start();
19         service.postConnect(42);
20         std::this_thread::sleep_for(
21             std::chrono::milliseconds(100));

```

```

22     service.stop();
23 }
24
25 // Верификация ПОСТФАКТУМ ---
26 // captured_id уже содержит реальное значение
27 ASSERT_NE(captured_id, -1);
28 Verify(Method(mockObserver, onConnected)
29     .Using(captured_id)).Exactly(1);
30
31 // Фаза 2: captured_id уже известен ---
32 // используем в стабе и верификации
33 mockObserver.ClearInvocationHistory();
34 mockTransport.ClearInvocationHistory();
35
36 When(Method(mockTransport, send)
37     .Using(captured_id, "hello")).Return(true);
38 When(Method(mockObserver, onDataReceived))
39     .AlwaysDo([](int, const std::string&) {});
40
41 {
42     AsyncService service(mockObserver.get(),
43                         mockTransport.get());
44     service.start();
45     service.postSend(captured_id, "hello");
46     std::this_thread::sleep_for(
47         std::chrono::milliseconds(100));
48     service.stop();
49 }
50
51 Verify(Method(mockObserver, onDataReceived)
52     .Using(captured_id,
53         std::string("ack:hello"))).Exactly(1);
54 }

```

На этапе верификации `captured_id` уже содержит реальное значение — `Verify` вызывается *после* выполнения. Однако на этапе стабирования значение по-прежнему должно быть известно заранее. Для динамического поведения — `AlwaysDo` с лямбдой. Разделение на две фазы с `ClearInvocationHistory` между ними позволяет использовать захваченное значение в стабах второй фазы.

Проблема П1 **частично решена** (верификация — да, стаб — нет без дополнительных лямбд).

Замечание. Аналогичный двухфазный подход (захватить значение в пер-

вой фазе, использовать во второй) возможен и в GMock –через SaveArg с последующим EXPECT_CALL. Принципиальное отличие FakeIt в том, что ClearInvocationHistory очищает только записанные вызовы, не затрагивая стабы, тогда как GMock-овский VerifyAndClearExpectations уничтожает всё. Кроме того, постфактум-верификация через Verify позволяет использовать runtime-значение без его заморозки в матчере.

6.3.4. Проверка проблемы П2: гранулярность сброса

Listing 24: FakeIt: поэтапная верификация (П2)

```
1  TEST(FakeIt, VerifyAndClearIsAllOrNothing) {
2      Mock<IEventObserver> mockObserver;
3      Mock<ITransport> mockTransport;
4
5      // Стабы на весь тест
6      When(Method(mockTransport, connect))
7          .AlwaysReturn(true);
8      When(Method(mockTransport, send))
9          .AlwaysReturn(true);
10     Fake(Method(mockObserver, onConnected),
11          Method(mockObserver, onDataReceived),
12          Method(mockObserver, onError));
13
14     // Фаза 1: connect
15     {
16         AsyncService service(mockObserver.get(),
17                               mockTransport.get());
18         service.start();
19         service.postConnect(42);
20         std::this_thread::sleep_for(
21             std::chrono::milliseconds(100));
22         service.stop();
23     }
24
25     // Верификация фазы 1
26     Verify(Method(mockTransport, connect)
27         .Using(42)).Exactly(1);
28     Verify(Method(mockObserver, onConnected)
29         .Using(42)).Exactly(1);
30     Verify(Method(mockObserver, onError)).Never();
31
32     // Сброс вызовов стабы -- остаются!
33     mockObserver.ClearInvocationHistory();
```

```

34     mockTransport.ClearInvocationHistory();
35
36     // Фаза 2: send стабы-- попрежнему- действуют
37     {
38         AsyncService service(mockObserver.get(),
39                               mockTransport.get());
40         service.start();
41         service.postSend(42, "hello");
42         std::this_thread::sleep_for(
43             std::chrono::milliseconds(100));
44         service.stop();
45     }
46
47     // Видим ТОЛЬКО вызовы фазы 2
48     Verify(Method(mockTransport, send)
49             .Using(42, std::string("hello"))).Exactly(1);
50     Verify(Method(mockObserver, onDataReceived)
51             .Using(42,
52                   std::string("ack:hello"))).Exactly(1);
53     Verify(Method(mockObserver, onError)).Never();
54 }

```

`ClearInvocationHistory` очищает только записанные вызовы, не трогая стабы — это более гранулярно, чем `VerifyAndClearExpectations` в GMock. Стабы, заданные через `When(...)`, продолжают действовать для обеих фаз. Однако сбросить стаб для одного метода, сохранив остальные, нельзя — только `Reset()` для всего мока.

Проблема П2 **частично решена** (верификация поэтапная, стабы — всё или ничего).

6.3.5. Проверка проблемы П3: условное ветвление

Важно отметить, что в FakeIt **невозможно** построить тест, эквивалентный GMock-версии, где expectations объявляются *до* выполнения и тест падает при несовпадении ветки. FakeIt не имеет аналога `EXPECT_CALL(...).Times(N)` — стабы через `When(...)` не верифицируют количество вызовов, а `Verify` работает только постфактум. Поэтому сразу покажем постфактум-подход:

Listing 25: FakeIt: условное ветвление (П3) — постфактум-верификация

```

1  TEST(FakeIt, ConditionalBranch_PostFactum) {
2      Mock<IEventObserver> mockObserver;
3      Mock<ITransport> mockTransport;
4
5      // Транспорт решает вернуть ошибку

```

```

6      When(Method(mockTransport, connect))
7          .Return(false);
8      // Ставим ОБА пути иначе ( UnexpectedMethodCall)
9      Fake(Method(mockObserver, onConnected),
10           Method(mockObserver, onError));
11
12      {
13          AsyncService service(mockObserver.get(),
14                               mockTransport.get());
15          service.start();
16          service.postConnect(1);
17          std::this_thread::sleep_for(
18              std::chrono::milliseconds(100));
19          service.stop();
20      }
21
22      // Верификация ПОСТФАКТУМ: мы ЗНАЕМ,
23      // что connect вернул false
24      Verify(Method(mockObserver, onError)).Exactly(1);
25      Verify(Method(mockObserver, onConnected)).Never();
26  }

```

Тест проходит, потому что верификация происходит *после* выполнения, и мы уже знаем, какая ветка сработала. Однако если результат **не известен** заранее (как в исходной формулировке ПЗ), приходится выносить условную логику за пределы фреймворка:

Listing 26: FakeIt: ручная условная верификация

```

1  // Результат неизвестен заранее
2  std::atomic<bool> connect_succeeded{false};
3
4  When(Method(mockTransport, connect))
5      .AlwaysDo([&](int) -> bool {
6          bool result = false; // runtime решение -
7          connect_succeeded.store(result);
8          return result;
9      });
10
11  Fake(Method(mockObserver, onConnected),
12       Method(mockObserver, onError));
13
14  // ... запуск сервиса ...
15

```

```

16 // Верификация: if/else BNE фреймворка
17 if (connect_succeeded.load()) {
18     Verify(Method(mockObserver, onConnected))
19         .Exactly(1);
20     Verify(Method(mockObserver, onError)).Never();
21 } else {
22     Verify(Method(mockObserver, onError))
23         .Exactly(1);
24     Verify(Method(mockObserver, onConnected))
25         .Never();
26 }

```

Нет аналога `.When()`: невозможно декларативно сказать «если состояние X — ожидай Y». Условная верификация реализуется только вручную через `if/else` в коде теста.

Проблема ПЗ **частично решена** (spy-модель снимает необходимость предсказывать, но условная строгая верификация остаётся ручной).

6.3.6. Возможность реализации `.When()` в FakeIt

Архитектурно внедрение `.When()` в FakeIt сложнее, чем в GMock и Trompeloeil. Причины:

1. **Разделение stubbing и verification.** `.When()` должен работать и при стабировании (в `RecordedMethodBody::getInvocationHandlerForActualArgs`), и при верификации (в `SequenceVerificationExpectation`). Два места правки вместо одного.
2. **Отсутствие единого expectation-объекта.** В GMock и Trompeloeil expectation — цельный объект с матчером, действием и ограничением на количество вызовов. В FakeIt матчер+действие (stub) и матчер+количество (verify) — разные сущности.
3. **Верификация по записанным вызовам.** FakeIt хранит `_actualInvocations` и фильтрует их при `Verify`. Для `.When()` пришлось бы пометать каждый записанный вызов флагом «при каком состоянии произошёл» — это меняет модель данных.

Объём модификации оценивается в 40–60 строк в двух классах, что существенно больше, чем 15–20 строк в одном классе для Trompeloeil.

6.3.7. Ограничения FakeIt

Помимо нерешённых проблем, FakeIt имеет ряд архитектурных ограничений:

- **Только виртуальные методы** –манипуляция vtable не работает с не-виртуальными, финальными методами, свободными функциями.
- **Нет множественного/виртуального наследования** — `static_assert` при попытке использования.
- **Зависимость от ABI** –отдельные реализации для GCC/Clang и MSVC, нет гарантий для экзотических компиляторов.
- **Нет потокобезопасности** –в отличие от GMock и Trompeloil, FakeIt не имеет глобального мьютекса. Параллельный доступ к моку — неопределённое поведение.

6.3.8. Оценка по критериям

Критерий 1: решение технических проблем.

Проблема	Статус	Комментарий
П1	Частично	Verify постфактум –да, стабы –нет
П2	Частично	ClearInvocationHistory, но стабы – Reset() на всё
П3	Частично	Spy-модель снимает необходимость предсказывать, но условная верификация ручная

Ни одна из проблем не решена полностью. Парадигма Arrange-Act-Assert обеспечивает гибкость на этапе верификации, но на этапе стабирирования ограничения аналогичны GMock.

Критерий 2: порог входа. Синтаксис кардинально отличается от GMock:

- Нет макросов для объявления моков (плюс).
- Нет `EXPECT_CALL` –вместо этого `When(Method(...)) + Verify(Method(...))`.
- Парадигма stub-then-verify вместо expect-then-verify требует переосмысления подхода к написанию тестов.
- Нет аналога `InSequence` / `After()` в привычном виде.

Переобучение оценивается как **значительное**: более 5 дней для команды, привыкшей к GMock, с необходимостью переписать все существующие тесты и полностью менять парадигму тестирования в рамках проекта.

7. Заключение

В данной работе впервые сформулирована и систематически исследована проблема *runtime expectations* в контексте C++ mocking-фреймворков. Проблема была декомпозирована на три подтипа: замораживание матчеров по значению (П1), грубая гранулярность сброса expectations (П2) и невозможность условного ветвления (П3). Для каждого подтипа представлен воспроизводимый тест-кейс на базе асинхронного сервиса с рабочим потоком.

Был проведён сравнительный анализ трёх фреймворков (GMock, Trompeloeil, FakeIt) по двум критериям: степень решения технических проблем и порог входа для команды, знакомой с GMock. Результаты показали, что ни один существующий фреймворк не решает проблему полностью: Trompeloeil решает П1 и П2, но не П3; FakeIt частично решает все три на этапе верификации, но не стабирирования.

GMock, как стандарт индустрии, обеспечивает наиболее развитую экосистему и богатый набор матчеров, однако его модель «record-then-verify» с иммутабельными expectations делает невозможным описание runtime-зависимого поведения. **Trompeloeil** за счёт RAII-семантики expectations решает проблемы фиксации матчеров (П1) и гранулярности сброса (П2) на архитектурном уровне, однако условное ветвление (П3) остаётся нерешённым. **FakeIt** с его парадигмой Arrange-Act-Assert и spy-моделью обеспечивает гибкость на этапе постфактум-верификации, но не предоставляет механизма условного стабирирования.

Предложенное расширение `.When()` для **GMock** - добавление предиката к expectation - решает все три проблемы при минимальном объёме модификации. Аналогичное расширение для **Trompeloeil** потребовало бы ещё меньше усилий, что объясняется более модульной архитектурой. Для **FakeIt** реализовать что-то похожее так же можно, однако в парадигме данного фреймворка такое решение **не** будет выглядеть органично, поэтому его целесообразность его реализации ставится под сомнение.

С точки зрения практики, расширение для **GMock** представляется наиболее ценным, так как оно не требует миграции на другой фреймворк, полностью обратно совместимо и позволяет команде, уже использующей GMock, получить необходимую функциональность без переобучения.

Если же для нас важнее перспектива и гибкость в будущем, то для новых проектов точно стоит рассмотреть Trompeloeil, так как за счет более модульной структуре и наличия полноценного RAII, этот фреймворк кажется крайне удачным базовым вариантом, который по своей текущей функциональности уже не уступает GMock, а также предоставляет ощутимо более удобные способы для модификации под собственные нужды по необходимости.

Дальнейшие направления исследования включают: реализацию и интеграцию `When()` в GMock, анализ потокобезопасности предложенного расширения при параллельном доступе к expectations, а также исследование альтернативного подхода на основе конечного автомата для описания сложных сценариев поведения моков.

Список литературы

- [1] Google. *GoogleTest – Google Testing and Mocking Framework*. <https://github.com/google/googletest>, 2024.
- [2] Google. *GoogleTest v1.15.0 – Release*. <https://github.com/google/googletest/tree/v1.15.0>, 2024.
- [3] Google. *gMock for Dummies*. https://google.github.io/googletest/gmock_for_dummies.html, 2024.
- [4] Google. *gMock Cookbook*. https://google.github.io/googletest/gmock_cook_book.html, 2024.
- [5] Google. *gMock Cheat Sheet*. https://google.github.io/googletest/gmock_cheat_sheet.html, 2024.
- [6] Google. *gmock-spec-builders.h – исходный код*. <https://github.com/google/googletest/blob/v1.15.0/googlemock/include/gmock/gmock-spec-builders.h>, 2024.
- [7] Hedberg B. *Trompeloeil – Header only C++14 mocking framework*. <https://github.com/rollbear/trompeloeil>, 2024.
- [8] Hedberg B. *Trompeloeil Reference Documentation*. <https://github.com/rollbear/trompeloeil/blob/main/docs/reference.md>, 2024.
- [9] Hedberg B. *Trompeloeil CookBook*. <https://github.com/rollbear/trompeloeil/blob/main/docs/CookBook.md>, 2024.
- [10] Alon E. *FakeIt – C++ mocking made easy*. <https://github.com/eranpeer/FakeIt>, 2024.
- [11] Alon E. *FakeIt Quickstart*. <https://github.com/eranpeer/FakeIt/wiki/Quickstart>, 2024.
- [12] Fowler M. *Mocks Aren't Stubs*. <https://martinfowler.com/articles/mocksArentStubs.html>, 2007.

- [13] Freeman S., Pryce N. *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley, 2009.
- [14] ISO/IEC 14882:2017. *Programming languages – C++*. International Organization for Standardization, 2017.
- [15] ISO/IEC 14882:2020. *Programming languages – C++*. International Organization for Standardization, 2020.
- [16] Zhou J. et al. *FoundationDB: A Distributed Key-Value Store* // Proc. ACM SIGMOD, 2021. <https://www.foundationdb.org/files/fdb-paper.pdf>.
- [17] *Deterministic Simulation Testing for Rust*. <https://s2.dev/blog/dst>, 2024.